

M0 · SECURITY REVIEW

M0

M Extensions Review.

A security review of M Extensions Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
M0	M Extensions Review	Guardian	November 10 – December 7, 2025	December 15, 2025

§ 00

Contents.

28 pages — overview, confidence ranking, scope, 16 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 11 findings	P. 09
	Remediation Review · 5 findings	P. 22
§ 05	Disclaimer	P. 27
§ 06	About Guardian	P. 28

§ 01

Overview.

Guardian reviewed M Extensions Review across 2 review rounds. The final report includes 1 Medium, 5 Low, 10 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	m0-foundation/m0-evm-m-extensions
CLIENT	METHODOLOGY
M0	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
M Extensions Review	49cc37acbb21
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	fcaae0d107f
AUDIT WINDOW	
November 10 – December 7, 2025	
FINAL REPORT	
December 15, 2025	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	1	0	1	0	0
Low	5	0	2	0	3
Informational	10	0	7	0	3
All severities	16	0	10	0	6

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed M Extensions Review in `m0-foundation/m0-evm-m-extensions`. The scope includes 191 source lines across 3 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
common/src/libs/TransferHelper.sol	32	58
src/components/pausable/Pausable.sol	17	44
src/projects/jmi/JMIExtension.sol	142	343

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Trusted Routers Cannot Execute Actions	DoS	MEDIUM	ACKNOWLEDGED	p. 9
L-01	Missing _beforeApprove Overwrite	Validation	LOW	ACKNOWLEDGED	p. 10
L-02	Asset Donations Can Lead To Underflow	DoS	LOW	RESOLVED	p. 11
L-03	_replaceAssetWithM and _wrapCalc Can Round To 0	Validation	LOW	RESOLVED	p. 13
L-04	No Extension Validation	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 14
L-05	MToken Can Be Wrapped	Unexpected Behavior	LOW	RESOLVED	p. 15
I-01	Arbitrage Opportunity On Depeg	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-02	replaceAssetWithM Can Be Misused	Warning	INFORMATIONAL	RESOLVED	p. 18
I-03	Precision Loss On Transfers Can Be Improved	Rounding	INFORMATIONAL	ACKNOWLEDGED	p. 19
I-04	Approvals For Frozen Account Cannot Be Cleared	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 20
I-05	Recipient Of Replaced Assets May Be Frozen	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 21
REMEDIATION REVIEW					
I-01	Decimals Can't Be Refetched	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-02	Accidental Asset Transfers Are Lost Now	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 23
I-03	canSwapViaPath() May Revert	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 24
I-04	Unnecessary Assignments	Best Practices	INFORMATIONAL	RESOLVED	p. 25
I-05	Unsafe uint240 Casting	Informational	INFORMATIONAL	RESOLVED	p. 26

MEDIUM **M-01**

ACKNOWLEDGED

Trusted Routers Cannot Execute Actions

CATEGORY	DoS
LOCATION	SwapFacility.sol
REVIEW	Main Review

DESCRIPTION

Whenever a function guarded with the `isNotLocked` modifier is called, `Locker.set(caller_)` is executed. If `msg.sender` is a trusted router, `caller_` is set to whatever address the router returns. Otherwise, the real `msg.sender` is used.

However, `SwapFacility` uses `msg.sender` in all of its functions. For example, it will try transferring assets out of `msg.sender`, etc... This will result in reverts when the function is called by a trusted router since it's expected to only forward the call, not pay for the transactions. Furthermore, `msg.sender` is passed to the `permit()` calls as well, meaning it has to be the signer of the message, which is impossible for the router.

There is also asymmetry existing when calling `JMI.wrap()`, because it uses `ISwapFacility(msg.sender).msgSender()`, which will be the correct forwarded sender.

RECOMMENDATION

Replace the `msg.sender` usage in `SwapFacility` with `msgSender()`.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Missing `_beforeApprove` Overwrite

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/feat/jmi/src/projects/jmi/JMIExtension.sol>

REVIEW Main Review

DESCRIPTION

The `JMIExtension` overwrites all the `MYieldTo0ne` hooks which reverts if a given account is frozen to also add the `_requireNotPaused` check, except for the `_beforeApprove` hook.

RECOMMENDATION

Consider to also overwrite and add the `_requireNotPaused` to the `_beforeApprove` hook.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

Asset Donations Can Lead To Underflow

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/49cc37acbb21d414dd64a8116504f4f747cc1c84/src/projects/jmi/JMIExtension.sol#L295-L298
REVIEW	Main Review

DESCRIPTION

The `JMIExtension` contract uses `balanceOf` to perform checks related to the asset amounts deposited instead of an internal mapping. On the other hand it uses the variable `totalAssets` to store the total amount of all deposited non-M-token assets.

This can lead to DoS and stuck funds if someone would transfer assets into the contract directly. For example:

- `totalAssets` = 0
- Two assets are whitelisted (Asset1 & Asset2)
- Bob deposits 100 Asset1 and 100 Asset2
- `totalAssets` = 200
- Eve transfers 50 Asset1 into the contract
- Eve calls `replaceAssetWithM` to get 150 Asset1 out of the contract
- `totalAssets` = 200 - 150 = 50
- Now it's not possible to get the 100 Asset2 out of the contract as `totalAssets` would underflow in that case

Another point is that asset caps can be consumed without wrapping.

For a wrap to succeed, the asset cap of the asset being wrapped should not be exceeded.

```
assetCap(asset) >= (IERC20(asset).balanceOf(address(this)) + amount)
```

Because `balanceOf()` is used, anyone can transfer asset tokens to the contract and consume the cap. The tokens cannot be utilized since `totalAssets` won't be modified.

RECOMMENDATION

Consider to implement an internal mapping instead to track asset amounts.

RESOLUTION

Status: Resolved.

L-02 Asset Donations Can Lead To Underflow

CONTINUED

<https://github.com/m0-foundation/m0-evm-m-extensions/pull/15>

LOW **L-03**

RESOLVED

_replaceAssetWithM and _wrapCalc Can Round To 0

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/49cc37acbb21d414dd64a8116504f4f747cc1c84/src/projects/jmi/JMIExtension.sol#L293>

REVIEW Main Review

— DESCRIPTION

The `_fromExtensionToAssetAmount` calculation in the `_replaceAssetWithM` function could round to 0 in the edge case that the given asset has less decimals than the \$M token and a dust amount is given.

This would result in the user losing \$M tokens and decreasing the `totalAssets` without receiving any assets in return.

The same behavior is also present in the `_wrap()` flow, where users can send assets, but have 0 extension tokens minted for them.

```
uint256 jmiAmount_ = _fromAssetToExtensionAmount(asset, amount);  
...  
_mint(recipient, jmiAmount_);
```

— RECOMMENDATION

Consider reverting in both cases if the resulting amount is 0.

— RESOLUTION

Status: Resolved.

<https://github.com/m0-foundation/m0-evm-m-extensions/pull/10>

LOW **L-04**

ACKNOWLEDGED

No Extension Validation

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/49cc37acbb21d414dd64a8116504f4f747cc1c84/src/swap/SwapFacility.sol#L499
REVIEW	Main Review

DESCRIPTION

Before interacting with the system, an extension has to be either approved by an admin or be an approved earner.

```
function isApprovedExtension(address extension) public view returns (bool) {  
    return _isApprovedEarner(extension) || isAdminApprovedExtension(extension);  
}
```

The `_isApprovedEarner()` function will return `true` for any address if the earners list is being ignored.

```
function _isApprovedEarner(address extension) private view returns (bool) {  
    return  
    IRegistrarLike(registrar).get(EARNERS_LIST_IGNORED_KEY) != bytes32(0) ||  
    IRegistrarLike(registrar).listContains(EARNERS_LIST_NAME, extension);  
}
```

Therefore, whenever the earners list is ignored, any arbitrary address can be used for an extension. This allows emitting `Swapped`, `SwappedOutM` and `JMIAssetReplaced()` events with arbitrary addresses and poses risks if the code changes in future.

RECOMMENDATION

Consider excluding the `EARNERS_LIST_IGNORED_KEY` part from `_isApprovedEarner()` and managing the swappers via approvals.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

MToken Can Be Wrapped

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/49cc37acbb21d414dd64a8116504f4f747cc1c84/src/swap/SwapFacility.sol#L285>

REVIEW Main Review

DESCRIPTION

To wrap an `mToken` into `JMIExtension` users must call `SwapFacility.swap()` and specify `mToken` as `tokenIn` and `JMIExtension` as `tokenOut`. This will execute the first branch in `_swap()`:

```
if (tokenIn == mToken) return _swapInM(tokenOut, amount, recipient);
```

The `_swapInM()` function is not adjusted to support `JMIExtension`, instead it used the generic `MExtension` interface to call `wrap()`

```
IMExtension(extensionOut).wrap(recipient, amount);
```

Because `JMIExtension` inherits from `MExtension`, the call will succeed, but `MExtension.wrap()` will be executed instead of `JMIExtension.wrap()`. There the internal `_wrap(address,address,uint256)` function is executed

```
function wrap(address recipient, uint256 amount) external onlySwapFacility {  
    // NOTE: 'msg.sender' is always SwapFacility contract.  
    // 'ISwapFacility.msgSender()' is used to ensure that the original caller is passed to  
    '_beforeWrap'.  
    _wrap(ISwapFacility(msg.sender).msgSender(), recipient, amount);  
}
```

This function is different than the `_wrap(address,address,address,uint256)` function defined in `JMIExtension`. From there on, the whole wrap is executed by the logic defined in `MExtension` instead of `JMIExtension`, which means the empty `_beforeWrap()` function of `MExtension` is executed and the following validation, including the pause check, is not performed:

```
function _beforeWrap(address asset, address account, address recipient, uint256 amount)  
internal view virtual {  
    _requireNotPaused();  
    if (!isAllowedAsset(asset)) revert AssetNotAllowed(asset);  
    if (!isAllowedToWrap(asset, amount)) revert AssetCapReached(asset);
```

L-05 MToken Can Be Wrapped

CONTINUED

```
super._beforeWrap(account, recipient, amount);  
}
```

Since `asset` here is `mToken`, skipping the two if statements is fine, but not calling `_requireNotPaused()` means the code doesn't respect the `paused` flag at all.

In result, wrapping mTokens into the `JMIExtension` will be possible even when it's in a paused state.

— RECOMMENDATION

Call the correct function if `tokenOut` in `SwapFacility.swapInM()` is the `JMIExtension`.

PROOF OF CONCEPT <https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/wrapDuringPausePOC/test/JMI.t.sol>

— RESOLUTION

Status: Resolved.

<https://github.com/m0-foundation/m0-evm-m-extensions/pull/9>

INFORMATIONAL **I-01**

ACKNOWLEDGED

Arbitrage Opportunity On Depeg

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/feat/jmi/src/projects/jmi/JMIExtension.sol
REVIEW	Main Review

DESCRIPTION

If one of the allowed assets (stablecoins) in the `JMIExtension` would lose in value a arbitrage opportunity arises:

- Traders buys assets for \$0.9
- Trader deposit the assets into the `JMIExtension`
- Trader swaps the `JMIExtension` into a safe one
- Repeat

RECOMMENDATION

Be aware of this risk and react as quickly as possible in that case by pausing the contracts and setting the cap of the given asset to zero.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

RESOLVED

replaceAssetWithM Can Be Misused

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/49cc37acbb21d414dd64a8116504f4f747cc1c84/src/swap/SwapFacility.sol#L123-L131
REVIEW	Main Review

DESCRIPTION

The `replaceAssetWithM` flow does not check if the given asset was whitelisted (`cap > 0`).

This function can therefore be used to swap any M0 extension to any token in the `JMIExtension`.

If by accident a token with higher value would land inside the contract any user could claim it by for example performing a \$1 M0 Extension against 1 WETH swap.

RECOMMENDATION

Be aware that this function can be used in that way and document this behavior.

Or consider to only allow the withdraw of whitelisted tokens to decrease potential attack surface. In this case however a new whitelist would make more sense than the current cap logic as otherwise it would no longer be possible to prevent further deposits while still allowing withdraws.

RESOLUTION

Status: Resolved.

<https://github.com/m0-foundation/m0-evm-m-extensions/pull/15>

INFORMATIONAL I-03

ACKNOWLEDGED

Precision Loss On Transfers Can Be Improved

CATEGORY Rounding
LOCATION Global
REVIEW Main Review

DESCRIPTION

Whenever an extension is wrapping mTokens, it pulls `amount` of these mTokens from the sender and mints the exact same amount of extension tokens. However, due to the index tracking behavior of the `MToken`, transferring `X` amount can result in less than `X` tokens received.

```
IMTokenLike(mToken).transferFrom(msg.sender, address(this), amount);  
_mint(recipient, amount);
```

In the case of `wrap()`, the system will mint more extension tokens than the received mTokens, which means some of the extension tokens will not be redeemable. The same is true for `_replaceAssetWithM()`.

The opposite rounding problem exists in `_unwrap()`, there the amount being subtracted is rounded up and because of that the system will be potentially losing value with each transfer.

RECOMMENDATION

There are already comments in the `MExtension` acknowledging the problem. While it's hard to be solved for the `_unwrap()` case, you can add balance delta tracking for `wrap()` and `_replaceAssetWithM()` to avoid some of the losses.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Approvals For Frozen Account Cannot Be Cleared

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/49cc37acbb21d414dd64a8116504f4f747cc1c84/src/projects/yeildTo0ne/MYieldTo0ne.sol#L168
REVIEW	Main Review

— DESCRIPTION

The approve hook reverts if the spender is frozen, regardless of the allowance amount. This prevents users from setting allowance to zero for spenders that later became frozen, effectively trapping their prior approvals until the spender is unfrozen.

— RECOMMENDATION

Allow `approve(owner, spender, 0)` to succeed even if the spender is frozen.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Recipient Of Replaced Assets May Be Frozen

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/49cc37acbb21d414dd64a8116504f4f747cc1c84/src/projects/jmi/JMIExtension.sol#L285-L307>

REVIEW Main Review

DESCRIPTION

The `JMIExtension` enforces freeze checks for wrap/unwrap/transfer via `MYieldToOne` hooks, but `_replaceAssetWithM()` does not validate whether the recipient is frozen. A frozen account can therefore receive allowed assets directly through `replaceAssetsWithM()`

RECOMMENDATION

If this is not expected, revert if the recipient is frozen.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Decimals Can't Be Refetched

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/fcaaee0d107f61dd2e8b445d83d89716f87c23e9/src/projects/jmi/JMIExtension.sol#L180
REVIEW	Remediation Review

DESCRIPTION

The `setAssetCap` saves the given assets decimals and it is not possible to update them in case the decimals of the given ERC20 token would ever change.

RECOMMENDATION

Be aware of that and consider to refetch the decimals every time the `setAssetCap` function is called to be able to react on such a change.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-02**

ACKNOWLEDGED

Accidental Asset Transfers Are Lost Now

CATEGORY Warning
LOCATION JMIExtension
REVIEW Remediation Review

— DESCRIPTION

Now asset balances are tracked with a internal balance. Therefore if now non \$M token assets are accidentally transferred into the `JMIExtension` contract they are lost.

— RECOMMENDATION

Consider to add a sweep function to be able to withdraw these assets.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

canSwapViaPath() May Revert

CATEGORY Warning

LOCATION <https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/fcaaee0d107f61dd2e8b445d83d89716f87c23e9/src/swap/SwapFacility.sol#L263-L268>

REVIEW Remediation Review

DESCRIPTION

A `try/catch` was added to `SwapFacility.canSwapViaPath()` to fetch the paused status of the two tokens.

```
// If contracts are paused, return false
try Pausable(tokenIn).paused() returns (bool tokenInPaused) {
  isTokenInPaused = tokenInPaused;
} catch {}
try Pausable(tokenOut).paused() returns (bool tokenOutPaused) {
  isTokenOutPaused = tokenOutPaused;
} catch {}
```

Because the addresses are arbitrary, any contract (or EOA with code) can be passed as a token, return a data that cannot be decoded to a `bool` and make the call to `canSwapViaPath()` to revert. This can lead to DOS for external integrators that expect the function to never revert.

RECOMMENDATION

Document the risk of reverting.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

RESOLVED

Unnecessary Assignments

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/m0-evm-m-extensions-team1-1762519293452/blob/fcaaee0d107f61dd2e8b445d83d89716f87c23e9/src/swap/SwapFacility.sol#L256-L257>

REVIEW Remediation Review

DESCRIPTION

The following two variables in the `SwapFacility.canSwapViaPath()` are assigned their default values (false for bool), which is unnecessary.

```
bool isTokenInPaused = false;  
bool isTokenOutPaused = false;
```

RECOMMENDATION

Consider removing the assignments.

RESOLUTION

Status: Resolved.

<https://github.com/m0-foundation/m0-evm-m-extensions/pull/26>

INFORMATIONAL I-05

RESOLVED

Unsafe uint240 Casting

CATEGORY	Informational
LOCATION	JMIExtension.sol
REVIEW	Remediation Review

DESCRIPTION

The balance of the tokens in the `Asset` struct is stored as `uint240`. The comment states that M token's supply won't exceed `uint240`, so balance should be safe.

```
// M token's supply can't exceed uint240, so uint240 is safe to use.  
uint240 balance;
```

However, this balance doesn't track M assets, but every other enabled asset. In the `_wrap()` function the `amount` of tokens transferred is unsafely cast to `uint240`. If the token is non-standard, i.e. has very big precision, and the asset cap allows it, the amount used may exceed `uint240` and result in wrong accounting.

RECOMMENDATION

1. Fix the comment to explain the right reason why using `uint240` is safe
2. Make sure to not use assets which can have such big supplies
3. Configure asset caps with this in mind

RESOLUTION

Status: Resolved.

<https://github.com/m0-foundation/m0-evm-m-extensions/pull/25>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.